

Augmented Offense — Module 2 — MIFARE Classic 1k gen2 · forge the UID

Len Noe & Dr. Gregory Carpenter · CW PENSEC

~23 min (0:41–1:04)

Module 2, for complete beginners

You have never touched any of this hardware? Perfect. This guide assumes that. Follow it top to bottom. Do exactly what each numbered step says, and check the green “✅ **You should see**” box before moving on. If something is wrong, the “⚠️ **If not**” box tells you what to do.

Implant analog: xMagic. **Time:** ~23 minutes.

1. What are you about to do, in plain English?

A **MIFARE Classic** card is the plastic access badge millions of offices use to open doors. Every card announces a serial number called a **UID**. Door systems often trust that UID as “who you are.”

The problem: the UID is **not a secret and not tamper-proof**. With a special blank card (a “**magic**” / “**gen2**” card), you can *write your own chosen UID onto it* — so the blank now pretends to be any badge number you want. That is the entire attack, and you’re about to do it with a number you pick yourself.

You are **not** copying anyone’s real badge. You pick a UID, write it to *your own* blank card, and read it back to prove it worked. Same skill, zero harm.

2. Mini-glossary (read once)

- **UID** — the card’s serial number. It’s either **8 hex characters** (a “**4-byte**” card) or **14 hex characters** (a “**7-byte**” card). “Hex” = digits 0–9 and letters A–F. Step 0 shows how to tell which one you have — it matters, because the number you write must be

the same length as your card.

- **Block 0** — the spot on the card where the UID lives. Normal cards lock it. **Magic / gen2** cards let you write it. Our blanks are gen2.
- **Key** — a 12-hex password that unlocks a section of the card so you can read it. A fresh blank uses common factory keys that ship in the repo — you don't need to make one.
- **Dump** — a full copy of everything on a card.
- **Reader** — the device that talks to the card. You'll use **one** of: **Proxmark3** (a small board on a USB cable), **Flipper Zero** (a handheld with a screen), or an **Android phone** with the right app.

iPhone users: iPhones *cannot* do this module — Apple blocks it. Use the pod's Proxmark3 or Flipper instead. That's normal, not a failure.

3. What you need at your seat

- ☐ A **Flipper Zero** (works on its own) *or* a **Proxmark3 + laptop** (ask a pod lead if you need a Flipper)
- ☐ Your **blank gen2 MIFARE Classic card** (a pod lead hands you this)
- ☐ The pod **demo reader** (to test your finished card)
- ☐ (*Proxmark path only*) a laptop with this project's repo open in a terminal

4. Step 0 — find your card's UID length, then pick a UID

MIFARE cards come in **two UID lengths**, and the number you write **must be the same length as your card**, or the write fails. So: first find out which card you have, then pick a matching number.

4a. Is your card 4-byte or 7-byte?

The quickest answer: **ask your pod lead** which blanks they handed out. If you'd rather check it yourself, read the card once and count:

- **Proxmark:** put the card on the round antenna, type `hf 14a reader`, look at the `UID:` line — **8 hex characters = 4-byte, 14 hex = 7-byte**.
- **Flipper: NFC** → **Read**, hold the card — the UID shows on screen. **4 pairs = 4-byte, 7 pairs = 7-byte**.

4b. Pick a UID of the matching length

Any value works — you’re proving the number isn’t a secret. Just match the length:

- **4-byte card** → **pick 8 hex characters**, e.g. `DECAF123` or `A1B2C3D4`.
- **7-byte card** → **pick 14 hex characters**, e.g. `04DECAF1234567`.

Make one up (using 0–9 and A–F), or let the Flipper generate one for you. **Write your chosen UID on a sticky note** and note whether it’s 4- or 7-byte.

Reading the card back later uses common factory keys that ship in the repo (`proxmark/common_keys.dic` for Proxmark, `android/mct_common_keys.dic` for the phone). A fresh blank uses those by default — you don’t need to set a key.

5. Pick your path

- Pod has a **Proxmark3** → **Path A**
- Pod has a **Flipper Zero** → **Path B**
- You have an **Android phone** and want to try the phone way → **Path C** (bonus)

You only need to complete **one** path.

Path A — Proxmark3 (step by step)

The Proxmark3 is a small circuit board on a USB cable. The flat round coil area is the **HF antenna** — that’s where the card goes.

A1. Connect and open the console

1. Plug the Proxmark3 into the laptop’s USB (your pod lead may have done this).
2. In the terminal, type `pm3` and press Enter. (If that says “not found,” ask the pod lead for the exact command — sometimes it’s `./pm3`.)
3. Wait a few seconds.

✓ **You should see** a prompt that looks like:

```
[usb] pm3 -->
```

That `[usb]` means the Proxmark is connected and listening. Every command below is typed at this `pm3 -->` prompt.

⚠ **If not:** if it says `[offline]`, the board isn't talking to the laptop. Unplug and replug the USB, try a different USB port, then run `pm3` again.

A2. Confirm it's alive (optional sanity check)

Type: `hw status` → you'll see hardware info scroll by. That confirms it works.

A3. Place your blank card on the antenna

Lay the **gen2 blank** flat on the round HF antenna, covering the coil. Keep it there for the next commands. Don't move it around.

A4. Forge your UID onto the card

Type this, replacing `0D301573` with **your** UID from the sticky note (8 hex for a 4-byte card, 14 hex for a 7-byte card — it must match, from Step 0):

```
hf mf csetuid -u 0D301573
```

Press Enter. *(The command is the same for both lengths — the length of the UID you type is what tells it which kind of card you have.)*

✅ **You should see** confirmation lines showing an **old UID** and a **new UID**, where the new UID matches yours. That means it wrote successfully.

⚠ **If not:** - “Can't select card” / “no tag found” → the card isn't seated. Re-center it on the antenna and run the command again. - “not a magic card” → you're holding the wrong card. Confirm it's the **gen2 blank**, not a locked card. Ask your pod lead.

A5. Read it back with the common keys

Type exactly:

```
hf mf chk --1k -f proxmark/common_keys.dic
```

This tries the common factory keys (shipped in the repo) against the card.

✅ **You should see** a table of sectors with keys found (lots of green/ `FOUND`). That proves the card is readable with the common keys.

A6. Confirm the UID

Type: `hf 14a reader`

✅ **You should see** a line `UID : <your UID>`. If it matches your sticky note, **you cloned it.** 🎉

A7. Validate on the demo reader

Tap your card on the pod **demo reader**. If it reacts the same as a known-good card, you're done. Tell your pod lead so they can move you along.

Path B — Flipper Zero (step by step)

The Flipper Zero is a handheld with a screen and arrow buttons. The **NFC antenna is under the top of the back** — that's where the card touches.

Buttons: the round center button is **OK/Select**. The arrows are **Up/Down/ Left/Right**. The **Back** button returns to the previous screen.

Menu wording differs slightly between Flipper firmware versions. If a label below doesn't match exactly, pick the closest one — the flow is the same.

B1. Open NFC

1. Press **OK** to open the main menu.
2. Arrow to **NFC** and press **OK**.

B2. Create a card with your UID

1. Choose **Add Manually**.
2. Scroll to the Mifare Classic 1K entry that **matches your card's UID length**: pick the **4-byte** option for a 4-byte card, or the **7-byte** option for a 7-byte card (from Step 0). Press **OK**. *(If the Flipper only offers one, that's the length it will make — match your sticky note to it.)*
3. When prompted for the UID, use the arrows to enter **your UID** from the sticky note (Left/Right to move, Up/Down to change a character). Confirm.
4. Choose **Save**, give it a name like `myclone`, and save.

✅ **You should see** your new saved card listed with your UID shown on its detail screen.

B3. Write it onto the blank

1. From **NFC** → **Saved**, open `myclone`.
2. Choose **Write** (on some firmware: **Write to Initial Card**).
3. Hold the **gen2 blank flat against the top-back** of the Flipper and keep it still.

✅ You should see “**Successfully written**” (or similar).

⚠️ **If not:** “Write failed” usually means the card slipped. Re-position it on the top-back of the Flipper and try again. Make sure it’s the **gen2 blank**.

B4. Read it back to confirm

1. From **NFC**, choose **Read**.
2. Hold the same card to the top-back again.

✅ You should see the card read with **your UID**. That’s the clone confirmed. Now tap it on the pod **demo reader** to validate.

Path C — Android phone (bonus, NXP-chipset phones only)

Some Android phones can do this natively. Many can’t (it depends on the phone’s NFC chip). If it doesn’t work, that’s expected — use Path A or B.

C1. One-time setup

1. Install “**MIFARE Classic Tool**” (MCT) from the Play Store.
2. Turn **NFC on**: Settings → search “NFC” → enable.
3. Put the repo’s key file where MCT can see it: copy `android/mct_common_keys.dic` onto the phone into the `MIFARE Classic Tool/key-files/` folder (a pod lead can help). You’ll pick it as your dictionary when reading the card back.

4-byte cards only on the phone. MCT’s block-0 write is easy for **4-byte** cards. If Step 0 said your card is **7-byte**, use the **Proxmark (Path A)** or **Flipper (Path B)** instead — the phone path gets fiddly there.

C2. Write your UID

1. Open **MCT**. If it says NFC is off, enable it.
2. Tap **Write Tag**.
3. Choose the **Write Block 0 (for magic/gen2 cards)** option.
4. Enter **your chosen 8-hex UID** (from the sticky note) when asked.
5. Hold the **gen2 blank flat against the back** of the phone (near the top for most phones) and keep it still until it says done.

✅ You should see a success message.

C3. Read it back

1. Tap **Read Tag**, select the `mct_common_keys` dictionary.
2. Hold the card to the back again.

✅ **You should see** your UID and the card's sectors. Validate on the demo reader.

⚠️ **If not:** "Tag not supported" / nothing happens → your phone's chip can't do MIFARE Classic. Switch to Path A or B. Not your fault.

6. How do I know I succeeded?

All three paths end the same way: **read the card back and see your own UID**, then **the pod demo reader reacts to it**. If both are true, you forged a working credential. Flag your pod lead.

7. Why it matters

You just wrote a badge number of your choosing onto a blank card, and a reader believed it. A real attacker does the exact same write — they just read the victim's number off a card in a hallway first. The "badge number" was never a secret. An implant (like an xMagic) does this straight from a hand: the badge gets copied in an elevator.

Defensive angle: UID ≠ identity. Retire bare MIFARE Classic. Move to challenge-response / identity-bound credentials — which is exactly what Module 4 (VivoKey) shows.

8. Troubleshooting (all paths)

What you see	What it means	What to do
Nothing happens when card touches reader	card not close enough / wrong spot	re-center the card on the antenna (Proxmark round coil / Flipper top-back / phone top-back), hold still
“No tag found” / “can’t select card”	reader can’t see the card	re-seat it, remove other cards/phones nearby, retry
“Not a magic card” / write refused	wrong card in hand	confirm it’s the gen2 blank ; ask pod lead
Reads are flaky, comes and goes	the DEF CON floor is RF-noisy at 13.56 MHz	move the reader away from neighbors; use the Faraday pouch on the front table to isolate; retry
Android app: “tag not supported”	phone chip can’t do MIFARE Classic	use Proxmark (Path A) or Flipper (Path B)
Proxmark prompt says [offline]	board not connected	replug USB, different port, run pm3 again
Numbers don’t match on read-back	typo when entering the UID	re-check your chosen UID on the sticky note, redo the write
Write refused / rejected / “wrong length”	your UID length doesn’t match the card	4-byte card needs an 8-hex UID; 7-byte card needs a 14-hex UID (Step 0). Fix the length and retry

9. Safety (non-negotiable)

- Write **only** the UID you generated, onto **your own** blank card.
- **Never** read, clone, or write anyone else’s card, phone, or a real building badge — not even “just to see.”
- These skills are illegal against systems you don’t own or aren’t authorized to test. Here, in this room, on these blanks, you’re clear. Outside, you’re not.

Presenters

- Len Noe (213) — <https://www.linkedin.com/in/len-noe/>

- **Dr. Gregory Carpenter (JunkBond)** — <https://www.linkedin.com/in/gcarpenter-cw-pensec/>